

How Strong Should the Shield Be?

Stress-Testing Defenses Against RAG Knowledge Poisoning

Aquib Raza Raja[†], *Student Member IEEE* and Muhammad Naveed Aman[†], *Senior Member IEEE*

[†]*School of Computing, University of Nebraska-Lincoln, Lincoln, NE, USA.*

Emails: araja2@huskers.unl.edu; naveed.aman@unl.edu

Abstract—[TODO: Write last, against the final result set.] The abstract must (i) lead with the stress-testing methodology contribution rather than with an attack result, (ii) name the four defense families studied and label the two proxies as proxies, (iii) state the evidence ceiling explicitly (oracle-level and development-set scale for the text-mutation results), and (iv) avoid any benchmark-wide rate.

Index Terms—retrieval-augmented generation, knowledge poisoning, defense evaluation, adversarial robustness, data integrity, trustworthy AI

I. INTRODUCTION

Motivation. [TODO: RAG adoption in production question answering and the resulting knowledge poisoning risk; corpus contents are now part of the trust boundary [1], [2].]

Gap. [TODO: Published defenses are evaluated almost exclusively against the attack distribution they were designed for, at the operating point their authors chose. A defense’s *stated* assumption and its *operative* assumption are not the same object, and only an assumption-targeted stress operator distinguishes them.]

Thesis. RAG poisoning defenses rest on different assumptions about how poisoned passages differ from benign ones: cross-passage embedding clustering, lexical and semantic density, learned feature separability, or answer aggregation. Stress tests constructed to target each assumption directly reveal distinct failure modes and distinct tradeoffs, and no single defense family is uniformly robust.

Contributions.

- 1) A defense-assumption stress-testing framework for poisoned RAG that separates oracle-space fragility from text-realizable evasion, from retrieval-preserving evasion, from answer-level outcome.
- 2) An oracle perturbation analysis exposing RAGDefender failure boundaries under the configuration we evaluate, in which pair-frequency structure, rather than global distribution matching alone, accounts for the observed failures. [Completed: oracle result] [Needs artifact verification: keep the configuration qualifier, or generalize the claim once the paper-faithful configuration is rerun through the oracle sequence]

- 3) Targeted text mutation families that stress distinct defense assumptions, with a measured asymmetry in which assumptions are text-reachable. [Completed: development-set result]
- 4) A two-regime evaluation of those mutations under fixed retrieved context and under full-retrieval replacement. [Completed: development-set result]
- 5) RobustRAG-KW, a generation-time isolate-then-aggregate proxy, evaluated as an *orthogonal* defense layer rather than as a stronger one. [Completed: RobustRAG-KW shortlisted scale-up]
- 6) Design lessons for future RAG poisoning defenses and for how such defenses should be evaluated.

[Pending: larger GPT confirmation set] and [Pending: toolkit ablation] — the introduction must state, in the contribution list itself, which contributions are currently development-scale. Do not defer this to Section VIII.

Roadmap. [TODO: One short paragraph mapping Sections II–IX.]

II. BACKGROUND AND RELATED WORK

A. Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) answers a query in two stages: a retriever selects a small set of passages from a corpus, and a generator conditions on them when producing its answer [1]. Retrieval is typically dense, embedding query and passages in a shared space and returning the top- k passages by inner product or cosine similarity [3], [4]. What makes this attractive — answers follow from editing the corpus rather than the model — is also what creates the attack surface studied here. Corpora are often large, externally sourced, or only semi-trusted, so a party able to write into one may influence generated answers without access to model weights, the prompt template, or the retriever, provided the inserted content is retrieved. Retrieved context is an untrusted input channel, and corpus contents are part of the trust boundary.

B. Knowledge Poisoning and PoisonedRAG

Knowledge poisoning targets that channel: the attacker inserts or rewrites a small number of passages so that retrieval brings poisoned evidence into the context of a target query

and the generator produces a chosen incorrect answer. PoisonedRAG states the two conditions a poisoned passage must meet — a *retrieval* condition, remaining similar enough to the target query to enter the top- k , and a *generation* condition, supporting the target answer strongly enough for the generator to adopt it — under a poison budget of N authored passages per target query that is small relative to the corpus, and in a black-box variant requiring no gradient access [2]. Related work optimizes poison directly against dense retrievers by discrete token search or in continuous embedding space [5], [6]. We do not propose a stronger attack: PoisonedRAG-style poison is the substrate, and what the defenses built against it actually depend on is the object of study.

C. Passage-Filtering Defenses

A broad class of defenses acts between retrieval and generation: inspect the retrieved set, discard passages that look adversarial, and pass the rest to an unmodified generator. PoisonedRAG evaluates four defenses in this spirit — paraphrasing, perplexity-based detection, duplicate-text filtering, and knowledge expansion, the last of which enlarges the retrieved set rather than pruning it [2]. Three families studied here belong to this class: RAGDefender, which reads cross-passage embedding similarity structure within the retrieved set [7]; FilterRAG, which scores each passage independently by a lexical and semantic density statistic; and its learned ML-FilterRAG variant, which replaces that fixed statistic with a classifier over density and fluency features [8]. Section III gives the mechanisms. What matters here is the shape of the commitment: each filter reduces detection to one measurable signal assumed to separate poisoned from benign passages — clustering or pairwise similarity structure, lexical and semantic overlap with the query and candidate answer, or separability in a learned feature space — and a set-level signal is relative, so it needs benign passages to be present in the retrieved set, whereas a per-passage signal does not.

D. Generation-Time and Aggregation-Based Defenses

A second class acts at generation rather than at passage admission. Isolate-then-aggregate defenses query the generator separately on each retrieved passage and aggregate the isolated responses, so that a bounded number of corrupted passages can influence only a bounded number of them; RobustRAG builds certifiable robustness on this structure [9]. The mechanism differs in kind from filtering: no passage need be identified as adversarial or removed, and the decision moves from which evidence is admitted to how independently derived answers are combined. This gives an orthogonal defense profile rather than a uniformly stronger one, since aggregation can hold where a filter fails, can fail when the isolated responses agree on the poisoned answer, and can abstain when the vote is contested, trading answer coverage for safety. Our RobustRAG-KW implementation is inspired by RobustRAG-style isolate-then-aggregate logic but is only a keyword/vote *proxy*; it is not a certified reproduction of

RobustRAG, and no certification result is claimed for it anywhere in this paper.

E. Robustness Evaluation and Stress Testing

Defenses of both kinds are usually reported as benchmark averages, against the attack distribution they were designed for, at an operating point their authors chose. Such a number does not identify which assumption a decision rule operationally depends on, and a stated assumption need not be the operative one. Defense-assumption stress testing instead constructs an operator against each named assumption and measures whether the rule survives it. Oracle perturbations act in the space the rule reads, where distribution-matching tools such as CORAL and MMD give well-defined ways to move one group of representations toward another [10], [11] and where the clustering-robustness literature reports that cluster assignments can be altered by small targeted perturbations that are not themselves outliers, albeit outside a retrieval setting [12], [13]; text-space operators must instead yield real passages, for which the semantics-preserving attack literature supplies the constraint vocabulary of semantic preservation, grammaticality, masked-LM fluency, and sentence-encoder cosine bounds [14]–[16]. Evidence does not transfer between them, so we separate four regimes (Sec. IV-A): oracle embedding and feature perturbation, testing which assumption a rule depends on; fixed retrieved context with rewritten poison, testing whether that assumption is reachable by real text; full-retrieval replacement, testing whether the rewrite still retrieves; and answer-generation attack success, testing whether the target answer is produced. A result in one regime licenses no conclusion in the next.

F. Positioning of This Work

This paper is a defense-assumption stress test, not simply a new attack paper. We do not claim a stronger attack than PoisonedRAG; we claim a way to find out what a defense actually depends on, and we apply it to four defense families. What prior evaluations do not separate is oracle fragility, text-realizable evasion, retrieval-preserving evasion, and answer-level outcome on the same defenses and the same queries, and without that separation a failure cannot be attributed to the assumption that produced it. The contribution is a characterization of defense failure modes and the design lessons that follow, not a universal attack and not a new certified defense.

III. THREAT MODEL AND DEFENSES UNDER STUDY

A. Threat Model

System. The system under study is a RAG pipeline: a dense retriever returns the top- k passages of a corpus for a given query, and a generator answers that query conditioned on the retrieved passages under a fixed prompt template. Everything the defense may inspect is derived from the query and from that retrieved set.

Attacker capabilities and goal. The attacker inserts or replaces a small number of passages in the retrieval corpus and

authors their text directly. It is black-box with respect to the generator and controls neither retrieval, nor the defense implementation, nor the defense’s thresholds. Its goal is targeted rather than degradational: for a chosen target query it fixes one specific incorrect answer, which it knows at authoring time along with the query, and it succeeds only when the generator produces that answer. A poisoned passage must therefore meet both of the PoisonedRAG conditions — be retrieved for the target query, and support the target answer strongly enough to be adopted [2].

Defender capabilities. The defender sees the query and the retrieved passages and has no poison labels at inference time. It acts either post-retrieval, deciding which retrieved passages reach the generator (RAGDefender, FilterRAG, ML-FilterRAG), or at generation time, controlling how an answer is formed from those passages (RobustRAG-KW). Which of the two applies is a property of the defense family and not something the attacker influences.

Poison budget and retrieval setting. We author $N = 5$ poisoned passages per target query, the PoisonedRAG default, and use retrieval depth $k = 10$ as the setting for the stress-testing regimes. Sec. V-A gives the rationale for both values, including why the poison-saturated $k = N$ regime is reported as a nominal condition rather than used as the stress-testing setting.

Fixed context and full-retrieval replacement. Mutations are evaluated in two regimes, defined in Sec. IV-A. Under fixed context the retrieved top- k membership and order are held constant by construction and only the poisoned passages’ text changes. Under full-retrieval replacement the mutated passages *replace* the original poisoned passages in the corpus and retrieval is rerun, so original and mutated poison are never both present in a context; duplicating the poison would inflate every downstream count and would make retrieval a different experiment from the one we intend to run. We preserve the self-query poison budget at five passages per target. When the shared adversarial pool retrieves poison associated with another query, we count it separately as cross-query poison rather than treating it as part of the target attack budget. **[Needs artifact verification: the per-origin accounting of Sec. IV-G is in place and one cross-query poison case is documented; the per-family counts reported in Sec. VI-D still need to be recounted from the origin audit]**

Scope. The object of study is defense robustness under stress testing, not attack strength. We do not claim a universal or stronger attack: our mutations are stress-test operators designed to probe defense assumptions, so a result licenses a statement about the assumption it targets and the regime it was measured in, and nothing broader.

B. Defenses Under Study

We study four defenses spanning the two layers of Sec. II. Each is described here only as far as its decision rule and the assumption that rule encodes; implementations and operating points are given in Sec. V.

1) *RAGDefender:* RAGDefender is a post-retrieval filter that reads pairwise structure among the embeddings of the retrieved passages [7]. A first stage estimates how many retrieved passages are adversarial, $\hat{N}_{adv}$, from per-passage mean and median pairwise cosine similarity within the retrieved set. A second stage accumulates, for each passage, how often it appears among the $\binom{\hat{N}_{adv}}{2}$ most similar pairs, and removes the $\hat{N}_{adv}$ passages with the highest resulting pair-frequency score. The operative assumption is that poisoned passages form detectable similarity structure relative to clean passages — concretely, that they are the densest high-similarity subgraph of the retrieved set — and because that signal is relative rather than per-passage, it is most meaningful when benign retrieved passages are present as contrast. Under the evaluated configuration, our stress tests probe whether pair-frequency structure remains discriminative when poisoned passages are moved toward cleaner neighborhood structure. Our oracle analysis is scoped to the RAGDefender configuration evaluated in our experiments; we do not claim that every RAGDefender variant fails for the same mechanism without rerunning the corresponding configuration. **[Needs artifact verification: state which RAGDefender configuration produced the oracle results of Sec. VI-B; a paper-faithful configuration exists and has not been rerun through the full oracle sequence]**

2) *FilterRAG:* FilterRAG is a post-retrieval filter that scores each passage independently by a semantic frequency-density statistic [8]. A small language model produces a candidate answer for the query; the query and answer terms are matched into the passage, and the summed frequency of matched terms is normalized by the passage’s unique-word count. A passage whose density exceeds a threshold ϵ is removed. The operative assumption is that poisoned passages reveal themselves through concentrated overlap with the question and the candidate answer terms used by the filter, relative to their own length. Unlike RAGDefender, the statistic is per-passage, so it does not require benign passages as a contrast set. We use the semantic matching mode as the paper-faithful FilterRAG setting and treat exact matching as an ablation. **[Completed: paper-faithful semantic matching mode implemented]**

3) *ML-FilterRAG:* ML-FilterRAG replaces that fixed threshold with a learned passage classifier over four features [8]: the FilterRAG density score, the matched-keyword frequency sum, passage perplexity, and the log-probability the small model assigns to its own answer. A retrieved passage is removed before generation when the classifier’s poison probability reaches t . The operative assumption is feature separability — that poisoned and clean passages occupy distinguishable regions of that feature space — and because density is one of the four features, a rewrite that lowers FilterRAG’s statistic is expected to transfer here. Our implementation is an *ML-FilterRAG-top-k proxy*: it scores the retrieved top- k set rather than reproducing the original top- s , filter, then top- k pipeline, and it is not a complete reproduction

of every setting of the published system.

4) *RobustRAG-KW*: RobustRAG-KW acts at generation time rather than at passage admission. It isolates each retrieved passage into its own generator call, extracts and normalizes a short answer from each, aggregates the isolated answers by keyword vote, and abstains when the vote is contested. The mechanism differs in kind from filtering because no passage is identified as adversarial or removed, which gives it a different exposure profile: a rewrite that lowers a density statistic may also weaken an isolated passage’s persuasive power, so aggregation can hold on cases where a filter is evaded, while a context in which most isolated passages independently support the attacker’s answer produces poison consensus and defeats the vote, and a contested vote yields abstention rather than an answer. The operative assumption is that clean evidence can contest the poison vote. RobustRAG-KW is a RobustRAG-inspired keyword/vote proxy used to study the generation-time aggregation paradigm; it is not a certified reproduction of RobustRAG [9]. It supports a claim about *where* in the pipeline a defense acts — an orthogonal generation-time defense profile — not a superiority claim over the passage filters.

C. Defense Assumptions

Table I states, for each family, the layer it acts at, the signal its decision rule reads, the assumption that signal encodes, and the stress-test pressure this paper applies to that assumption; Fig. 2 gives the same relation as a conceptual map. The four families are separated by layer and by signal rather than by strength, and each stress operator is built against one named assumption rather than against a defense as a whole.

Three caveats bound how the table may be read, and belong here rather than only in Sec. VIII. ML-FilterRAG is an ML-FilterRAG-top- k proxy and RobustRAG-KW is a RobustRAG-inspired keyword/vote proxy, so an outcome for either constrains the paradigm we implemented and not the published system. Oracle pressures act on embeddings or features in a frozen space and are not text-realizable by themselves: no perturbed vector has been shown to correspond to a real passage. And each pressure is assumption-targeted by construction, so a defense left unmoved by its own pressure is evidence about what that operator reaches, not evidence of robustness in general.

IV. STRESS-TESTING METHODOLOGY

A. Overview and Evaluation Regimes

Fig. 1 shows the pipeline: poison authoring, retrieval, defense, generation, and metrics. Each experiment starts from a target query and the 5 PoisonedRAG passages authored for it; a *stress-test operator* then modifies either those passages’ text or the embeddings and features a defense computes from them, and an unmodified defense implementation scores the result. Each operator enters at exactly one stage, and the entry point determines what its result can license. We record whether poisoned passages are removed before generation, survive retrieval, or reach the generator and drive the answer.

TABLE I
DEFENSE FAMILIES, THE SIGNAL EACH DECISION RULE READS, THE ASSUMPTION THAT SIGNAL ENCODES, AND THE STRESS PRESSURE APPLIED IN THIS PAPER.

Defense	Layer	Signal	Key assumption	Stress-test pressure
RAGDefender	Post-retrieval	Pairwise embedding structure (set-level)	Poison forms a detectable similarity concentration	Disperse poison toward clean neighborhood structure
FilterRAG	Post-retrieval	Semantic query/answer-term density (per passage)	Poison overlaps concentratedly with query and answer evidence	Low-overlap answer support
ML-FilterRAG (proxy)	Learned post-retrieval	Density, matched terms, perplexity, answer likelihood	Poison and clean are feature-separable	Clean-reference feature shift; transfer from low overlap
RobustRAG-KW (proxy)	Generation-time aggregation	Isolated per-passages answer votes	Clean evidence can contest the poison vote	Poison consensus; isolated persuasion

Retrieved poison is attributed by origin: *self-query poison* was authored for the query under test, while *cross-query poison* was authored for a different query of the shared adversarial pool and never counts against the target’s budget. Text-space conditions are *replacement-only mutations*, so the budget stays at $N = 5$ throughout. The mutation work is split into a *development set*, on which the rewrite prompts were designed, and a held-out *confirmation set* (Sec. IV-F); operating points, models, and metric definitions are deferred to Sec. V.

Table II defines the four regimes in which those operators are evaluated. *Oracle embedding and feature stress tests* modify poison representations directly in the space a decision rule reads, with retrieval membership and passage text held fixed; they establish which statistical assumption the rule operatively depends on, and establish no attack. *Fixed-context text mutation* rewrites poison text into a retrieved context whose membership and order are fixed by construction; it establishes that the assumption is reachable by a real passage once poison is already retrieved, and nothing about retrieval. *Full-retrieval replacement* substitutes mutated for original poison in the corpus and reruns retrieval under a preserved budget, establishing that the rewrite still satisfies the retrieval condition. *Answer-generation evaluation* runs generation on the post-defense context and scores the target incorrect answer, establishing an answer-level outcome for the cases evaluated and only for those.

The regimes are ordered by realism but not by value: oracle tests expose boundary conditions, fixed-context tests isolate defense sensitivity, full-retrieval replacement tests retrievability, and answer-generation tests downstream impact. They answer different questions and are reported separately; collapsing them into one metric would discard the distinction the framework exists to draw.

TABLE II
EVALUATION REGIMES, WHAT EACH CONTROLS, WHAT EACH TESTS, AND STATUS.

Regime	What is controlled	What it tests	Status
Oracle	Text and retrieval fixed; embeddings or features perturbed	Which assumption the decision rule depends on	Completed (RAGDefender, ML-FilterRAG proxy)
Fixed context	Top- k membership and order fixed; poison text rewritten	Whether the assumption is reachable by a real passage	Completed on the development set (6 queries)
Full retrieval	Corpus substitution, replacement only, budget preserved	Whether rewritten poison survives rerun retrieval	Completed on development-set bundles
Answer generation	Post-defense context passed to the generator	Whether the attacker answer is produced, or abstention occurs	Completed on shortlisted cases

B. Oracle Perturbations

Oracle perturbations are not passage rewrites. They are diagnostic interventions that act on a frozen space to identify which geometric or feature signals a defense depends on.

E1 clean-anchor interpolation. Each poisoned embedding is interpolated toward an assigned clean anchor, $\mathbf{z}' = \alpha \mathbf{z}_{\text{poison}} + (1 - \alpha) \mathbf{z}_{\text{anchor}}$, under four anchor-assignment strategies — rank-aligned, nearest-bijection, farthest-bijection, and random — and a sweep of α from 1.0 to 0.3.

Distribution matching. MMD minimization performs kernel mean matching of the poisoned group toward the clean group [11] under a preservation penalty bounding how far embeddings may move; CORAL-PCA and CORAL-ridge align covariances with poisoned embeddings as source and clean embeddings as target [10], in a reduced-rank and a regularized form respectively.

Feature-space oracle. The ML-FilterRAG proxy’s four features are interpolated toward clean targets, from a nearest-clean bijection to the global clean centroid, putting the same locality question to a learned rule.

These are not text-space attacks and not learned domain-adaptation models; they are formal embedding-space stress tests of a similarity-based decision rule. For RAGDefender, the oracle analysis is scoped to the evaluated configuration; paper-faithful reruns may re-attribute the dominant failure mechanism.

C. Targeted Text Mutation Families

We define three hand-authored GPT rewrite families, one per targeted assumption, each rewriting all 5 poisoned passages of a query while preserving the attacker’s target wrong answer. The mutation families are stress-test operators designed to probe defense assumptions.

RAGDefender-targeted. Diversify the discourse frame of each passage and reduce poison-poison semantic similarity, so the poison set stops being the densest high-similarity subgraph and the pairwise clustering signal weakens.

FilterRAG-targeted. Reduce lexical and semantic overlap with the query and candidate-answer terms while preserving support for the same target wrong answer, and raise unique-word count, so the frequency-density statistic falls.

ML-FilterRAG-targeted. Make the passages read as ordinary clean reference text, using clean retrieved passages as stylistic references, so the learned feature profile shifts toward the clean distribution.

Rewrites are authored through fixed GPT prompt packets, one per family, so prompt design is itself a development-set artifact. The current 6-query mutation bundle is a development set used for prompt design and failure-mode discovery; a larger held-out confirmation set is planned before final claims (Sec. IV-F).

D. Fixed-Context and Full-Retrieval Evaluation

Under fixed context the retrieved top-10 membership and order are reconstructed verbatim from the pilot artifacts and only the 5 poisoned passages’ text differs between conditions, clean retrieved passages being byte-identical across every condition of a case. Holding the retriever out isolates a defense’s sensitivity to passage content from any change in what retrieval returns. We report removed poison, removed clean, the residual poison fraction, and the defense-specific diagnostic score each rule computes (Sec. V). Fixed-context success does not imply a practical corpus attack because retrieval is bypassed.

Full-retrieval replacement closes that gap: mutated passages *replace* the original poisoned documents in the adversarial corpus and retrieval is rerun at top- $k = 10$, so the regime tests the PoisonedRAG retrieval condition [2] instead of assuming it. Full-retrieval replacement preserves the self-query poison budget: original and mutated poisoned passages are never included together for the same target query. Per case we track self-query poison retrieved, cross-query poison retrieved, clean passages retrieved, and each defense’s removal outcome.

E. Answer Generation and RobustRAG-KW Evaluation

Answer generation is run on shortlisted cases with no defense and after each passage-filtering defense, so a defense’s effect on the produced answer is observed rather than inferred from removal counts. Strict ASR is a token-boundary match of the attacker’s target wrong answer in the generated text, and abstentions are detected by the same detector RobustRAG-KW uses, so a refusal and a correct answer are never merged into one outcome. Strict matching over-counts on yes/no and contrastive answers, a limitation stated with the reporting conventions of Sec. V and revisited in Sec. VIII.

RobustRAG-KW runs in the same regime but reads a different signal: it isolates each retrieved passage into its own generator call, extracts and normalizes a short answer from each, aggregates the isolated answers by keyword vote, and abstains when the vote is contested [9]. RobustRAG-KW is evaluated as a generation-time aggregation proxy; it is not treated as a passage filter and is not compared as a certified RobustRAG implementation, so it contributes a

mechanistically distinct generation-time profile rather than a stronger one. Because each case costs one generator call per retrieved passage plus an aggregation step, it is evaluated only on *shortlisted* cases and its outcomes are reported as counts, never rates. **[Do not overclaim: shortlist only]**

F. Confirmation-Set and Toolkit Protocols

[Pending: larger GPT confirmation set] The targeted mutation bundle above is development-scale evidence. The planned confirmation set is 20 held-out HotpotQA queries at $k = 10$ and $N = 5$, with 3 mutation families $\times$ 2 bundles $\times$ 5 passages, i.e. 600 mutated passages; identifiers are fixed in advance and disjoint from the development set, and the prompt packets are frozen before any confirmation-set case is evaluated. The prose of Sec. VI-C is written so that confirmation-set numbers substitute for development-set numbers without restructuring, the development-set numbers becoming prompt-design evidence.

[Pending: toolkit ablation] The toolkit mutation ablation follows the confirmation set and asks one narrow question: whether generic library perturbations [14]–[16] under a matched budget reproduce the per-defense effects of the defense-aware GPT rewrites. It is a control on the *specificity* of the targeted families, not a search for a stronger attack. **[TODO: Fix the toolkit, the constraint set, and the matched budget before running, so the comparison is a controlled substitution of the rewrite operator only.]**

G. Reproducibility Controls

Retrieval and defense scoring are cached, so a rerun reproduces prior numbers exactly rather than approximately, and answer generation is cached by (model, prompt) hash so that RobustRAG-KW aggregation and the reports derived from it can be regenerated from cached answers without new generator calls. GPT prompt packets are hashed and committed per mutation family; confirmation-set query identifiers are fixed in advance and held out; the poison budget is replacement-only, preserved at $N = 5$ in every mutated condition with no original-plus-mutated duplication; and self-query versus cross-query poison is accounted separately, so a case that retrieves poison authored for a different pool query is never counted as if all retrieved poison targeted the case at hand. Every evaluation script consumes committed artifacts and loads models read-only, and the test suite makes no API calls.

V. EXPERIMENTAL SETUP

A. Dataset, Retrieval, and Poisoning Budget

All text-level regimes share one dataset and one retriever: the HotpotQA test split in BEIR format [17] encoded with Contriever [4] under inner-product scoring. We use the PoisonedRAG black-box LM-targeted procedure [2] to construct poison over a shared 50-query pool, from which every query set below is drawn: 6 queries for the targeted-mutation development set, a 40/10 train/held-out split for the ML-FilterRAG-top- k proxy, 9 shortlisted cases for RobustRAG-KW answer generation, and 20 further held-out queries for

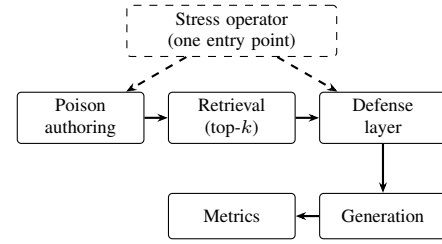


Fig. 1. Stress-testing pipeline. A stress operator enters at exactly one stage, and the entry point determines what its result licenses. **[TODO: Redraw with the four regimes of Table II marked as gates between stages.]**

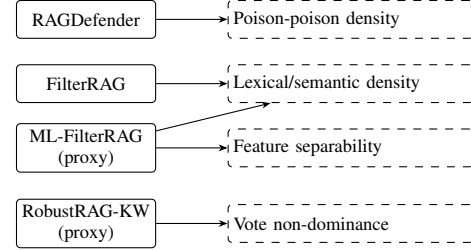


Fig. 2. Conceptual map from defense families to the assumptions their decision rules encode. The edge from ML-FilterRAG to lexical/semantic density is the shared-feature path that predicts transfer of low-overlap rewrites. **[TODO: Annotate each edge with the stress operator that targets it.]**

the planned confirmation set. **[Needs artifact verification: Before submission, mirror the 50-query pool and 40/10 ML-FilterRAG split manifest into a tracked artifact for reproducibility.]** We set $N = 5$ poisoned passages per target query, the PoisonedRAG default, and evaluate two depths. At $k = 5$ retrieved and injected counts coincide, a poison-saturated context reported as a nominal condition. At $k = 10$ the context is mixed, which PoisonedRAG’s ablation identifies as the point where clean passages enter while poison is still usually retrieved; it is therefore the stress-testing setting. Set-level defenses such as RAGDefender are most informative in the mixed regime, where poisoned structure can be contrasted against benign retrieved passages. Full-retrieval replacement preserves the self-query poison budget at $N = 5$ by replacement rather than addition.

B. Defense Implementations

RAGDefender is run in one configuration throughout, scoring Stage-1 and Stage-2 pairwise structure with a sentence-transformer encoder; every RAGDefender claim here is scoped to it. **[Needs artifact verification: the evaluated configuration is the legacy variant; a paper-faithful variant exists and has not been rerun through the oracle or mutation sequences]** *FilterRAG* uses semantic matching, the paper-faithful setting, with `flan-t5-small` producing the candidate answer and an all-MiniLM encoder matching query and answer terms into each passage; exact matching is a diagnostic ablation. *ML-FilterRAG* is an *ML-FilterRAG-top-k proxy*: a random forest over the four features of Sec. III-B — density score, matched-keyword frequency sum, passage perplexity under `distilgpt2`, and the small model’s log-probability

for its own answer — trained on 40 pool queries and evaluated on 10 held out, scoring the retrieved top- k set rather than the published top- s , filter, then top- k pipeline. *RobustRAG-KW* is a RobustRAG-inspired keyword/vote proxy [9]: each retrieved passage is isolated into its own generator call, a short answer is extracted and normalized, and the normalized answers are aggregated by vote. It is not a certified RobustRAG reproduction. Because each case costs one call per passage plus aggregation, it is run only on shortlisted cases.

C. Generation Setup

Answer generation uses gpt-3.5-turbo at temperature 0.1 with a 150-token limit and one fixed prompt template shared by every condition. **[Needs artifact verification: artifacts record the API model alias, not a dated snapshot]** Standard generation issues one call on the full post-defense context; RobustRAG-KW issues one isolated call per retrieved passage plus aggregation. Generation is cached by a (model, prompt) hash and the test suite makes no API calls (Sec. IV-G).

D. Development Set versus Confirmation Set

The 6-query targeted mutation work is the *development set*: the rewrite prompts were designed on it, so it supports failure-mode discovery rather than benchmark-wide rates and cannot serve as held-out confirmation. The planned *confirmation set* (Sec. IV-F) is 20 held-out HotpotQA queries at the same depth and budget, 3 families $\times$ 2 bundles $\times$ 5 passages, i.e. 600 mutated passages, with the prompt packets frozen beforehand. **[Pending: larger GPT confirmation set]**

E. Thresholds and Operating Points

FilterRAG uses the published density threshold $\epsilon = 0.2$ with a semantic matching threshold of 0.6, neither retuned for the proxy small LM. For the ML-FilterRAG-top- k proxy, $t = 0.4$ is the clean-preserving operating point, selected from the sweep of Table VII and held fixed in every cross-defense comparison. **[Completed: development-set result]** RobustRAG-KW uses its default rule: a winner must hold at least half of the non-abstaining vote, and a contested or tied vote yields abstention.

F. Metrics and Reporting Conventions

We report *removed poison* and *removed clean*, the counts of retrieved poisoned and clean passages a filter discards, and the *residual poison fraction*, which is measured over the surviving post-defense context, not over the originally retrieved top- k set. At the answer level we report *strict ASR*, a token-boundary match of the attacker’s target answer, and *abstention*, counted separately so a refusal is never merged with a correct answer; RobustRAG-KW adds the *wrong-answer* and *correct-answer vote shares* over the isolated per-passage answers. Retrieved poison is always reported split by origin into *self-query* and *cross-query poison* (Sec. IV-A). Strict matching overcounts on yes/no and contrastive answers, and an answer carrying both the strict target string and the correct answer

is adjudicated rather than counted automatically. RobustRAG-KW outcomes come from a shortlisted, filter-stress sample and are reported as counts, never rates. **[Do not overclaim: short-list only]** Development-set numbers are prompt-design and failure-mode evidence: confirmation-set evaluation is pending.

VI. RESULTS

A. RQ1: What assumptions do existing defenses rely on?

[TODO: Read Table I and Fig. 2 as a result, not as background: the four families are separated not by strength but by the layer they act at and the signal they read, and that separation is what makes their failure modes disjoint.]

B. RQ2: Can oracle perturbations expose RAGDefender failure boundaries?

Regime: oracle embedding space, text and retrieval membership held fixed. **[Completed: oracle result]**

E1 clean-anchor interpolation breaks all 6 of the 6 processable RAGDefender-success cases. MMD breaks them partially, and depends on the preservation penalty. CORAL-PCA and CORAL-ridge do not break them, despite reducing measured distribution distance and displacing poisoned embeddings further than the interventions that succeed. Under the configuration we evaluate, what accounts for these failures is therefore collapse of the Stage-2 top-pair and pair-frequency structure rather than global distribution matching: a defense can be moved a long way in distribution space without being moved at all in the direction its decision rule reads. **[TODO: State the query-strategy denominator explicitly next to the 6/6 figure, so a reader cannot mistake per-strategy units for per-query units.]** **[Needs artifact verification: the failure denominators reported across the oracle comparison artifacts are stated with different units (query-level versus query-strategy); recount from the batch CSV and fix one convention]** **[Needs artifact verification: these oracle results were produced against the legacy RAGDefender configuration; a paper-faithful configuration exists and has not been rerun through the full oracle sequence, so this subsection must either scope its claim to the evaluated configuration or wait for the rerun]**

This is oracle embedding-space evidence. It is not automatically a text-realizable attack, and Sec. VI-C shows that in this case it is not.

C. RQ3: Do targeted text rewrites weaken passage filters?

Regime: fixed retrieved context, poison text rewritten, development set of 6 queries. **[Completed: development-set result]**

Generic rewrites produce modest effects and serve as the negative control: rewriting per se does not evade these filters. Among the targeted families, the FilterRAG-targeted low-overlap family produces the strongest cross-defense effect, weakening FilterRAG and transferring to the ML-FilterRAG-top- k proxy; the ML-FilterRAG clean-reference family moves only its own target; and the RAGDefender-targeted family

TABLE III

ORACLE PERTURBATIONS AGAINST RAGDEFENDER ON THE 6 ORIGINALLY SUCCESSFUL DEVELOPMENT-SET CASES, UNDER THE CONFIGURATION WE EVALUATE. ORACLE-SPACE EVIDENCE ONLY.

Perturbation	Failures	Main effect	Interpretation
E1 clean-anchor	6/6	Poison-poison count collapses	Breaks the operative assumption directly
MMD	Partial [Needs artifact verification: count]	Collapse only at weak preservation	Breaks it when the penalty permits large movement
CORAL-PCA	0	Distribution distance falls; top-pair structure intact	Distribution matching is not sufficient
CORAL-ridge	0	Largest displacement, no failures	Displacement magnitude is not the operative variable

TABLE IV

FIXED-CONTEXT CROSS-DEFENSE EFFECT OF THE TARGETED MUTATION FAMILIES ON THE DEVELOPMENT SET. VALUES ARE THE MEAN CHANGE IN REMOVED POISON PASSAGES (OF 5) RELATIVE TO THE UNMUTATED CONTEXT; NEGATIVE MEANS THE DEFENSE REMOVED FEWER POISONED PASSAGES. 6 QUERIES, ONE BUNDLE PER QUERY AND FAMILY.

Mutation family	RAGD.	FR	ML ($t=0.4$)	Interpretation
RAGDefender-targeted	0.00	-0.17	-0.67	Intended target un-
FilterRAG-targeted	-0.33	-1.17	-2.17	Strongest defense effect cross- ^A
ML-FilterRAG-targeted	+0.17	0.00	-0.17	Narrow, own-target only

generic-rewrite control from an earlier pilot produced a smaller mean effect on the ML-FilterRAG proxy than any targeted family. [Needs artifact verification: restate the generic-control figure with its own query count, which differs from 6]

does not move RAGDefender at all. The asymmetry is the finding: the density-based assumptions are text-reachable under hand-authored rewriting and RAGDefender’s clustering assumption, which the oracle of Sec. VI-B breaks reliably, is not.

[Pending: larger GPT confirmation set] These are development-set numbers on the 6 queries whose rewrite prompts were designed against these same defenses. They are evidence that the mechanism is reachable, not an estimate of how often it is reachable.

D. RQ4: Do rewritten poison passages survive retrieval?

Regime: full-retrieval replacement, corpus substitution, budget preserved. [Completed: development-set result]

Replacement-only full retrieval was completed on 18 mutation-bundle-1 cases. In every case the 5 self-query mutated poison passages returned to the top-10, so the rewrites of Sec. VI-C do not purchase filter evasion at the cost of retrievability. A minority of cases additionally retrieved poison authored for a different pool query; those passages are accounted separately and are never counted as self-query poison. [Pending: larger GPT confirmation set] This is development-scale evidence, not benchmark-wide evidence, and it does not establish that retrieval survival is near-total in general.

TABLE V

FULL-RETRIEVAL REPLACEMENT ON THE DEVELOPMENT SET: 18 BUNDLES, REPLACEMENT ONLY, POISON BUDGET PRESERVED AT $N = 5$.

Mutation family	Cases	Self-query poison retrieved	Cross-query poison	Defense degradation
RAGDefender-targeted	6	5/5 all	[Needs artifact verification: count]	Reproduces fixed context
FilterRAG-targeted	6	5/5 all	[Needs artifact verification: count]	Strongest, as in fixed context
ML-FilterRAG-targeted	6	5/5 all	[Needs artifact verification: count]	Own target only

artifact verification: fill the per-family cross-query poison counts and the retrieved rank range from the full-retrieval artifacts; the aggregate retrieval-survival claim is verified, the per-family breakdown is not]

E. RQ5: Does generation-time aggregation change the outcome?

Regime: answer generation on shortlisted full-retrieval cases. [Completed: RobustRAG-KW shortlisted scale-up] [Do not overclaim: shortlist only]

Table VI reports the 9 shortlisted cases. RobustRAG-KW did not produce the attacker answer in 3 shortlisted cases where at least one passage-filtering defense did. Two of those three outcomes are abstentions and one is a clean defense, so the row counts of Def. and Abs. must be read separately: a refusal and a correct answer are not the same outcome for a user. RobustRAG-KW also failed on the cases where every isolated per-passage call independently voted for the poisoned answer, and it abstained on the majority of cases under contested votes.

The claim this supports is orthogonality, not superiority: aggregation reads a different signal than the passage filters, so it can hold where they fail and fail where they hold. The 9 cases were shortlisted for filter stress and are not a benchmark-wide sample; no rate may be computed from them.

F. RQ6: Do toolkit mutations reproduce the targeted GPT failures?

[Pending: toolkit ablation] [TODO: One paragraph, written after the confirmation set, reporting whether generic library perturbations under a matched budget reproduce the per-defense effects of the targeted GPT families. A negative result here strengthens the specificity claim of Sec. VI-C and should be reported as such.]

VII. DISCUSSION AND DESIGN LESSONS

A. Lesson 1: Clustering defenses depend on poison-poison structure

[TODO: Anchor: Sec. VI-B. The dependence is on the poison set being the densest subgraph, not on any

TABLE VI
ROBUSTRAG-KW GENERATION-TIME AGGREGATION COMPARED WITH PASSAGE-FILTERING DEFENSES ON SHORTLISTED FULL-RETRIEVAL MUTATION CASES.

Family	Query	Self/All poison	Passage filters	RRKW	s_{wrong}	s_{gold}	Reading
FR-tgt	Gibson/Zurrapapote gin	5/5	RD=ASR, FR=ASR, ML=ASR	ASR	0.90	0.10	Poison consensus
FR-tgt	Menges/Avakian occupation	5/5	RD=Def., FR=ASR, ML=ASR	ASR	0.50	0.00	Poison consensus
FR-tgt	Schmeichel IFFHS 1992	5/5	RD=Def., FR=Def., ML=Abs.	Abs.	0.50	0.20	Abstain (contested)
FR-tgt	Teide/Garajonay parks	5/5	RD=Def., FR=Def., ML=Def.	Abs.	0.30	0.40	Abstain (contested)
FR-tgt	Roth/Childers England	5/7	RD=Def., FR=Def., ML=ASR	Abs.	0.50	0.50	Abstain (contested)
FR-tgt	Ferocactus/Silene plants	5/6	RD=Def., FR=ASR, ML=ASR	Def.	0.20	0.80	Defends evasion
RD-tgt	Gibson/Zurrapapote gin	5/5	RD=Def., FR=ASR, ML=Def.	Abs.	0.90	0.10	Abstain (contested)
RD-tgt	Roth/Childers England	5/7	RD=Def., FR=Def., ML=Def.	Abs.	0.50	0.50	Abstain (contested)
ML-tgt	Roth/Childers England	5/7	RD=Def., FR=Def., ML=Def.	Abs.	0.50	0.50	Abstain (contested)

The 9 cases are a shortlisted, filter-stress sample, not a benchmark-wide rate. ASR means the target wrong answer was produced under strict matching; Def. means the target was not produced; Abs. denotes abstention. Self/All poison is self-query poison retrieved over all poison retrieved, so 5/7 marks a case that also retrieved cross-query poison.

```
PLACEHOLDER fig:mutation_transfer_heatmap
mutation family (rows) x defense (columns),
cell = mean change in removed poison
source: evaluation_normalized/
normalized_cross_defense_failure_matrix.csv
```

Fig. 3. Cross-defense transfer of the targeted mutation families on the development set. Rows are mutation families, columns are defenses, and cells are the mean change in removed poison passages. [TODO: Generate at publication resolution; consider folding into Table IV if the page budget tightens.]

```
PLACEHOLDER fig:retrieval_survival
per case: self-query mutated poison retrieved
in top- $k$ , with rank range;
cross-query poison marked separately
source: full_retrieval_pilot/
```

Fig. 4. Full-retrieval replacement survival on the development set: mutated self-query poison returning to the top-10 and its rank range, with cross-query poison marked separately. [TODO: Generate from the full-retrieval artifacts.]

```
PLACEHOLDER fig:robustrag_vote_distribution
per shortlisted case: wrong-answer vs
correct-answer vote share, with the
abstention band shaded
source: robustrag_kw_scaleup_
generation_results.csv
```

Fig. 5. RobustRAG-KW vote shares over the 9 shortlisted cases, with the contested region that produces abstention marked. Counts only; the shortlist does not support a rate. [Do not overclaim: shortlist only]

global property of the poison distribution, which is why distribution matching leaves the defense intact. This lesson inherits the configuration scope and the open verification of Sec. VI-B; write it only after that resolves, since a re-

TABLE VII
ML-FILTERRAG-TOP- k PROXY THRESHOLD SWEEP ON A HELD-OUT HOTPOTQA 50-QUERY DRY RUN (10 HELD-OUT QUERY IDS, 20 (q, k) RECORDS, 118 RETRIEVED POISONED AND 32 RETRIEVED CLEAN PASSAGES), WITH THE FILTERRAG SEMANTIC-THRESHOLD REFERENCE ROW. $t = 0.4$ IS THE CLEAN-PRESERVING OPERATING POINT USED THROUGHOUT.

Method / threshold	Poison recall	Clean FPR	Residual poison
FilterRAG semantic, $\epsilon = 0.2$	0.983	0.781	0.222
ML-FilterRAG, $t = 0.35$	0.958	0.219	0.167
ML-FilterRAG, $t = 0.4$	0.949	0.156	0.182
ML-FilterRAG, $t = 0.50$	0.915	0.156	0.270
ML-FilterRAG, $t = 0.90$	0.678	0.000	0.543

attribution there changes this lesson.]

B. Lesson 2: Overlap-based filters are vulnerable to low-overlap support

[TODO: Anchor: Sec. VI-C. A false claim can be supported in text that is not keyword-dense in the query or the answer.]

C. Lesson 3: Learned filters inherit their feature assumptions

[TODO: Anchor: Sec. VI-C and Fig. 2. The ML-FilterRAG proxy inherits frequency-density as a feature, so transfer from low-overlap rewrites is a predicted consequence of the shared feature rather than a coincidence.]

D. Lesson 4: Aggregation is orthogonal but abstention-heavy

[TODO: Anchor: Sec. VI-E. Aggregation can resist some filter-evasion cases, fails under poison consensus, and frequently abstains under contested votes. State the safety-versus-coverage tradeoff plainly and do not present abstention as a defense success.]

E. Lesson 5: Oracle vulnerability does not imply a text-realizable attack

[TODO: Anchor: Sec. VI-B versus Sec. VI-C. The operator that most reliably breaks RAGDefender in embedding space is the one whose text-space approximation moved RAGDefender least. Acceptable framing: RAGDefender is fragile under oracle embedding perturbation and harder to break with natural text rewrites, and we report that gap

rather than closing it.] [Pending: stronger RAGDefender text-space mutation operator]

F. Practical Implications for Future RAG Defenses

[TODO: Short paragraph: state assumptions explicitly and distinguish stated from operative; evaluate outside the attack distribution the defense was designed for; require retrieval preservation before claiming evasion and answer-level evidence before claiming an end-to-end outcome; report security and clean-context utility together, since a defense that empties the context has relocated the failure rather than fixed it; and combine layers that read different signals, because the failure modes documented here are disjoint.]

VIII. LIMITATIONS AND FUTURE WORK

Development-set size. The targeted mutation work covers 6 queries and 18 bundles, with one rewrite per poison slot per family, so per-family statistics are means over queries and not over repeated independent attempts.

HotpotQA-only current evidence. All stress-testing regimes use one dataset and one retriever; cross-dataset and cross-retriever generality is not measured.

Strict ASR limitations. Token-boundary matching overcounts, most visibly on yes/no answers, and a strict hit co-occurring with the correct answer requires adjudication.

RobustRAG-KW proxy limitation. A keyword/vote proxy, not certified RobustRAG; no certification result is claimed.

Answer-generation cost and shortlist bias. The 9 cases were shortlisted because passage filters were already under stress on them, so their outcome distribution is not that of the benchmark. [Do not overclaim: shortlist only]

Pending confirmation set. [Pending: larger GPT confirmation set] 20 held-out queries, 3 families, 2 bundles, 600 mutated passages, with the larger full-retrieval replacement run and the shortlisted answer generation on that set both pending.

Pending toolkit ablation. [Pending: toolkit ablation]

Additional datasets and defenses. [TODO: Name the next dataset and the next defense family, with the reason each was chosen.]

Stronger RAGDefender text-space mutations. [Pending: stronger RAGDefender text-space mutation operator] An operator with an explicit objective on the Stage-2 pair-frequency quantity, rather than on discourse diversity, is the natural next step; either outcome is informative, since success closes the gap of Lesson 5 and failure sharpens it.

Implementation fidelity. [Needs artifact verification: state the proxy substitutions and any deviation of each defense implementation from its published description, including which RAGDefender configuration produced the oracle results of Sec. VI-B]

IX. CONCLUSION

[TODO: Write last, in about four sentences. Restate the assumption-to-operator framework, the finding that

top-pair collapse rather than distribution matching drives the clustering defense’s failure, the asymmetry between text-reachable and text-unreachable assumptions, and the orthogonality of generation-time aggregation. No new claims.]

ACKNOWLEDGMENT

[TODO: Funding and computing acknowledgment placeholder.]

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [2] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” in *Proc. USENIX Security Symposium*, 2025.
- [3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [4] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, “Unsupervised dense information retrieval with contrastive learning,” *Transactions on Machine Learning Research*, 2022.
- [5] J. Su, P. Nakov, and C. Cardie, “Corpus poisoning via approximate greedy gradient descent,” in *Findings of the Association for Computational Linguistics: ACL 2025*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 4274–4294.
- [6] Y. Li, P. Eustratiadis, S. Lupart, and E. Kanoulas, “Unsupervised corpus poisoning attacks in continuous space for dense retrieval,” in *Proc. 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. Padua, Italy: ACM, 2025.
- [7] S. Kim *et al.*, “Rescuing the unpoisoned: Efficient defense against knowledge corruption attacks on RAG systems,” in *Proc. Annual Computer Security Applications Conference (ACSAC)*, 2025, vERIFY full author list and page range.
- [8] K. Edemacu *et al.*, “Defending against knowledge poisoning attacks during retrieval-augmented generation,” in *arXiv preprint*, 2025, vERIFY venue and full author list.
- [9] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal, “Certifiably robust RAG against retrieval corruption,” in *arXiv preprint*, 2024, vERIFY final venue.
- [10] B. Sun, J. Feng, and K. Saenko, “Return of frustratingly easy domain adaptation,” in *Proc. AAAI Conference on Artificial Intelligence*, 2016.
- [11] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, 2012.
- [12] A. Chhabra, A. Roy, and P. Mohapatra, “Suspicion-free adversarial attacks on clustering algorithms,” in *Proc. AAAI Conference on Artificial Intelligence*, vol. 34, no. 4, 2020, pp. 3625–3632.
- [13] A. E. Cinà, A. Torcinovich, and M. Pelillo, “A black-box adversarial attack for poisoning clustering,” *Pattern Recognition*, vol. 122, p. 108306, 2022.
- [14] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, “Is BERT really robust? a strong baseline for natural language attack on text classification and entailment,” in *Proc. AAAI Conference on Artificial Intelligence*, 2020.
- [15] L. Li, R. Ma, Q. Guo, X. Xue, and X. Qiu, “BERT-ATTACK: Adversarial attack against BERT using BERT,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [16] J. X. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, “TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP,” in *Proc. EMNLP: System Demonstrations*, 2020.
- [17] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.